

Convolution

FFT

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int) a.size() + b.size() + 1)
```

```

        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

FFT with mod

```

ll add(ll a, ll b) {
    return (a + b);
}

ll mul(ll a, ll b) {
    return (a * b);
}

// IF THERE IS MOD SPACE DO NOT FORGET THE DIVISION IN THE multiply
// FUNCTION!!!!!!!!!!!!!!!!!!!!!!

// Important notes => if size polynomial is 2^i (must be a power of 2)
// complexity multiplication is O(i * 2^i) == O(n logn)
// Power polynomial to p complexity is O(n logn logp) NOT LIKE NORMAL FFT
#define AND 0
#define OR 1
#define XOR 2
void fwht(vector<ll> &a, ll inv, ll f) {
    ll sz = a.size(); // sz must be (1 <= i)
    for (ll len = 1; 2 * len <= sz; len <= 1) {
        for (ll i = 0; i < sz; i += 2 * len) {
            for (ll j = 0; j < len; j++) {
                ll x = a[i + j];
                ll y = a[i + j + len];
                if (f == AND) {
                    if (!inv) a[i + j] = y, a[i + j + len] = add(x, y);
                    else a[i + j] = add(y, -x), a[i + j + len] = x;
                } else if (f == OR) {
                    if (!inv) a[i + j + len] = add(x, y);

```

```

        else a[i + j + len] = add(y, -x);
    } else if (f == XOR) {
        a[i + j] = add(x, y);
        a[i + j + len] = add(x, -y);
    }
}
}
}
}
vector<ll> multiply(vector<ll> a, vector<ll> b, ll f) {
    ll sz = a.size();
    fwht(a, 0, f);
    fwht(b, 0, f);
    vector<ll> c(sz);
    for (ll i = 0; i < sz; ++i) {
        c[i] = mul(a[i], b[i]);
    }
    fwht(c, 1, f);
    if (f == XOR) {
        for (ll i = 0; i < sz; ++i) {
            c[i] = c[i] / sz;
        }
    }
    return c;
}

```

FFT tricks

1. All Pairs Sum

To find the frequency of all possible sums $a_i + a_j$:

- **The Polynomial:** Let $P(x) = \sum freq[a_i] \cdot x^{a_i}$.
- **The Operation:** Calculate $R(x) = P(x) \cdot P(x)$.
- **The Result:** The coefficient of x^k in $R(x)$ is the number of pairs (i, j) such that $a_i + a_j = k$.
- **Note:** If $i \neq j$ is required, subtract the cases where an element is added to itself (x^{2a_i}) and divide by 2 for unordered pairs.

2. All Subarray Sums

A subarray sum $[l, r]$ is defined as $S_r - S_{l-1}$, where S is the prefix sum array.

- **The Polynomial:** This is actually an "All Pairs Difference" problem. Let $P(x) = \sum x^{S_i}$.
- **The Operation:** Multiply $P(x)$ by its "reverse," $P(x^{-1})$.

- **Handling Negatives:** To avoid negative exponents in FFT, shift the indices:

$$P_{rev}(x) = \sum x^{MAX_SUM - S_i}$$
- **The Result:** The coefficient of $x^{MAX_SUM + k}$ gives the count of subarrays summing to k .
- **Zero Sums:** Count these separately using a frequency map of prefix sums: $\sum \binom{freq[S_i]}{2}$.

3. All Pairs Multiply with Distance $d = i - j$

When you need to find $\sum a_i \cdot a_j$ where the indices have a specific relationship (like a fixed distance):

- **The Polynomials:** * $A(x) = \sum a_i x^i$
 - $B(x) = \sum a_i x^{-i}$ (Reverse of A)
- **The Result:** The coefficient of x^d in $A(x)B(x)$ corresponds to the sum of products $a_i \cdot a_j$ where $i - j = d$.
- **Cyclic Shifts:** To handle cyclic correlations, use **Periodic Convolution**. Double the second array ($A + A$) and multiply with the reversed first array to capture all wrap-around distances.

4. All Subset Sums (Knapsack)

To find how many ways to form a sum using any subset of elements:

- **The Polynomial:** For each element v_i , create $P_i(x) = (1 + x^{v_i})$.
- **The Operation:** $R(x) = \prod_{i=1}^n (1 + x^{v_i})$.
- **Optimization:** Instead of n FFTs, group identical values. If value v appears k times, the contribution is $(1 + x^v)^k$.
- **Advanced:** Use $\ln$ and $\exp$ to turn the product into a sum: $\exp(\sum \ln(1 + x^{v_i}))$. This reduces the complexity to $O(W \log W)$ where W is the maximum possible sum.

String matching

- `ret[k]` = number of matching characters between `pat` and `s` shifted by `k`

```
vector<int> string_matching(string &s, string &pat) {
    int n = s.size(), m = pat.size();
    vector<int> poly1(n), poly2(m);
    vector<int> ret(n);
    int shift = m - 1;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < n; j++) {
            poly1[j] = ((s[j] - 'a') == i);
        }
        for (int j = 0; j < m; j++) {
            poly2[shift - j] = ((pat[j] - 'a') == i);
        }
    }
}
```

```

    }
    auto ans = multiply(poly1, poly2);
    for (int j = m-1; j < n; j++) {
        ret[j - (m-1)] += ans[j];
    }
}
return ret;
}

void solve(int tc) {
    string s;
    cin >> s;
    string pat = s;
    for (int i = 0; i < pat.size(); i++) s += '*';
    auto match = string_matching(s, pat);
    int n = s.size(), m = pat.size();
    int maxi = 0;
    for (int i = 1; i <= n - m; i++) {
        maxi = max(maxi, match[i]);
    }
    cout << maxi << endl;
    for (int i = 1; i <= n - m; i++) {
        if (match[i] == maxi) cout << i << " ";
    }
}

```

wild card

```

using cd = complex<double>;
const double PI = acos(-1), eps = 5e-4; // If you get a wrong answer you can
change the eps lower or higher till you pass

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }
}

```

```

for (int len = 2; len <= n; len <= 1) {
    double ang = 2 * PI / len * (invert ? -1 : 1);
    cd wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
        cd w(1);
        for (int j = 0; j < len / 2; j++) {
            cd u = a[i+j], v = a[i+j+len/2] * w;
            a[i+j] = u + v;
            a[i+j+len/2] = u - v;
            w *= wlen;
        }
    }
}

if (invert) {
    for (cd & x : a)
        x /= n;
}
}

vector<cd> multiply(vector<cd> const& a, vector<cd> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    return fa;
}

void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<cd> poly1(n), poly2(m);

    for (int i = 0; i < n; ++i) {

```

```

        double angle = 2*PI*(s[i]-'a')/26;
        poly1[i] = cd(cos(angle), sin(angle));
    }
    for (int i = 0; i < m; ++i) {
        if(patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
        else {
            double angle = 2*PI*(patt[m-i-1]-'a')/26;
            poly2[i] = cd(cos(angle), -sin(angle));
        }
    }

    vector<cd> ans = multiply(poly1, poly2);
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');

    int tot = 0;
    vector<int> pos;
    for (int i = 0; i < n; ++i) {
        if(fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}

```

Excat match

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)

```

```

        swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

void solve(int tc) {

    string s, patt; cin >> s >> patt;

```

```

int n = (int)s.length(), m = (int)patt.length();

vector<int> poly1(n), poly2(m);

vector<int> ans_match(n);

for (int i = 0; i < 26; ++i) {
    for (int j = 0; j < n; ++j) {
        poly1[j] = (s[j] - 'a') == i;
    }
    for (int j = 0; j < m; ++j) {
        poly2[j] = (patt[m-j-1] - 'a') == i;
    }
    vector<int> ans = multiply(poly1, poly2);
    for (int j = 0; j < n; ++j) {
        ans_match[j] += ans[m-1+j];
    }
}

int tot = 0;
vector<int> pos;
int wild_cnt = (int)count(patt.begin(), patt.end(), '*');
for (int i = 0; i < n; ++i) {
    if(ans_match[i] == m - wild_cnt) {
        ++tot;
        pos.push_back(i);
    }
}

cout << tot << "\n";
for(auto & p : pos) cout << p << " ";
cout << "\n";
}

```

Big int Multiply.txt

```

string mul_two_big_int(const string &s1, const string &s2) {
    int n = s1.size(), m = s2.size();

    vector<int> poly1(n), poly2(m);
    for (int i = 0; i < n; ++i) {
        poly1[n-i-1] = s1[i] - '0';
    }
}

```

```

    for (int i = 0; i < m; ++i) {
        poly2[m-i-1] = s2[i] - '0';
    }

    vector<int> ans = multiply(poly1, poly2);
    int k = ans.size();

    for (int i = 0; i < k - 1; ++i) {
        ans[i + 1] += ans[i] / 10;
        ans[i] = ans[i] % 10;
    }

    string final = to_string(ans[k - 1]);
    for (int i = k - 2; i >= 0; --i) {
        final += (char)(ans[i] + '0');
    }

    for (int i = 0; i < k; ++i) {
        if(final[i] != '0') return final.substr(i);
    }
    return "0";
}

string power_of_big_int(string s, int p) {
    string ans = "1";
    while (p) {
        if(p&1) ans = mul_two_big_int(ans, s);
        s = mul_two_big_int(s, s);
        p >>= 1;
    }
    return ans;
}'''

```

FWHT

```

ll add(ll a, ll b) {
    return (a + b);
}
ll mul(ll a, ll b) {
    return (a * b);
}

// IF THERE IS MOD SPACE DO NOT FORGET THE DIVISION IN THE multiply
FUNCTION!!!!!!!!!!!!!!!!!!!!!!

// Important notes => if size polynomial is 2^i (must be a power of 2)

```

```

// complexity multiplication is  $O(i * 2^i) == O(n \log n)$ 
// Power polynomial to p complexity is  $O(n \log n \log p)$  NOT LIKE NORMAL FFT
#define AND 0
#define OR 1
#define XOR 2
void fwht(vector<ll> &a, ll inv, ll f) {
    ll sz = a.size(); // sz must be  $(1 \ll i)$ 
    for (ll len = 1; 2 * len <= sz; len <= 1) {
        for (ll i = 0; i < sz; i += 2 * len) {
            for (ll j = 0; j < len; j++) {
                ll x = a[i + j];
                ll y = a[i + j + len];
                if (f == AND) {
                    if (!inv) a[i + j] = y, a[i + j + len] = add(x, y);
                    else a[i + j] = add(y, -x), a[i + j + len] = x;
                } else if (f == OR) {
                    if (!inv) a[i + j + len] = add(x, y);
                    else a[i + j + len] = add(y, -x);
                } else if (f == XOR) {
                    a[i + j] = add(x, y);
                    a[i + j + len] = add(x, -y);
                }
            }
        }
    }
}

vector<ll> multiply(vector<ll> a, vector<ll> b, ll f) {
    ll sz = a.size();
    fwht(a, 0, f);
    fwht(b, 0, f);
    vector<ll> c(sz);
    for (ll i = 0; i < sz; ++i) {
        c[i] = mul(a[i], b[i]);
    }
    fwht(c, 1, f);
    if (f == XOR) {
        for (ll i = 0; i < sz; ++i) {
            c[i] = c[i] / sz;
        }
    }
    return c;
}

```

```

template<int MOD>
struct FWHT {

```

```

int fast(int b, int e) {
    int res = 1;
    for (; e; e >>= 1, b = 1ll * b * b % MOD)
        if (e & 1)
            res = 1ll * res * b % MOD;
    return res;
}

inline int add(int x, int y) {
    return x + y - (x + y >= MOD ? MOD : 0);
}

inline int sub(int x, int y) {
    return x - y + (x - y < 0 ? MOD : 0);
}

void FST(vector<int> &a, bool inv) {
    for (int n = (int) a.size(), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step)
            for (int j = i; j < i + step; j++) {
                int &u = a[j], &v = a[j + step];
                tie(u, v) =
                    // inv ? pii(sub(v,u), u) : pii(v, add(u,v)); //
                    // inv ? pii(v, sub(u,v)) : pii(add(u,v), u); //
                pair<ll, ll>(add(u, v), sub(u,
v)); // XOR /// include-line
            }
        }
    if (inv) {
        int divisor = fast((int) a.size(), MOD - 2);
        for (int &x: a) x = 1ll * x * divisor % MOD; // XOR only ///
include-line
    }
}

vector<int> conv(vector<int> a, vector<int> b) {
    FST(a, 0);
    FST(b, 0);
    for (int i = 0; i < (int) a.size(); i++) a[i] = 1ll * a[i] * b[i] %
MOD;
    FST(a, 1);
    return a;
}
};

```

NTT

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}

// Primitive Root of the mod of form 2^a * b + 1
int generator () {
    vector<int> fact;
    int phi = mod-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=mod; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= modpow (res, phi / fact[i]) != 1;
        if (ok) return res;
    }
    return -1;
}

int modpow(int b, int e, int m) {
    int ans = 1;
    for (; e; b = (ll)b * b % m, e /= 2)
        if (e & 1) ans = (ll)ans * b % m;
    return ans;
}

void ntt(vector<int> &a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    vector<int> rt(2, 1); // erase the static if you want to use two moduli;
```

```

    for (int k = 2, s = 2; k < n; k *= 2, s++) { // erase the static if you
want to use two moduli;
        rt.resize(n);
        int z[] = {1, modpow(root, mod >> s, mod)};
        for (int i = k; i < 2*k; ++i) rt[i] = (ll)rt[i / 2] * z[i & 1] % mod;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; ++j) {
                int z = (ll)rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
        }
    }
}

vector<int> conv(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int s = (int)a.size() + (int)b.size() - 1, B = 32 - __builtin_clz(s), n =
1 << B;
    int inv = modpow(n, mod - 2, mod);
    vector<int> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod *
inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

ll CRT(ll a, ll m1, ll b, ll m2) {
    __int128 m = m1*m2;
    ll ans = a*m2%m*modpow(m2, m1-2, m1)%m + m1*b%m*modpow(m1, m2-2, m2)%m;
    return ans % m;
}

/*

int mod, root, desired_mod = 1000000007;
const int mod1 = 167772161;
const int mod2 = 469762049;
const int mod3 = 754974721;

```

```

const int root1 = 3;
const int root2 = 3;
const int root3 = 11;

int CRT(int a, int b, int c, int m1, int m2, int m3) {
    __int128 M = (__int128)m1*m2*m3;
    ll M1 = (ll)m2*m3;
    ll M2 = (ll)m1*m3;
    ll M3 = (ll)m2*m1;

    int M_1 = modpow(M1%m1, m1 - 2, m1);
    int M_2 = modpow(M2%m2, m2 - 2, m2);
    int M_3 = modpow(M3%m3, m3 - 2, m3);

    __int128 ans = (__int128)a*M1*M_1;
    ans += (__int128)b*M2*M_2;
    ans += (__int128)c*M3*M_3;

    return (ans % M) % desired_mod;
}

*/

```

NTT with any prime mode

```

#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, mod = 998244353;

struct base {
    double x, y;
    base() { x = y = 0; }
    base(double x, double y): x(x), y(y) { }
};

inline base operator + (base a, base b) { return base(a.x + b.x, a.y + b.y); }
inline base operator - (base a, base b) { return base(a.x - b.x, a.y - b.y); }
inline base operator * (base a, base b) { return base(a.x * b.x - a.y * b.y,
a.x * b.y + a.y * b.x); }
inline base conj(base a) { return base(a.x, -a.y); }
int lim = 1;
vector<base> roots = {{0, 0}, {1, 0}};
vector<int> rev = {0, 1};
const double PI = acosl(- 1.0);
void ensure_base(int p) {

```

```

    if(p <= lim) return;
    rev.resize(1 << p);
    for(int i = 0; i < (1 << p); i++) rev[i] = (rev[i >> 1] >> 1) + ((i & 1) <<
(p - 1));
    roots.resize(1 << p);
    while(lim < p) {
        double angle = 2 * PI / (1 << (lim + 1));
        for(int i = 1 << (lim - 1); i < (1 << lim); i++) {
            roots[i << 1] = roots[i];
            double angle_i = angle * (2 * i + 1 - (1 << lim));
            roots[(i << 1) + 1] = base(cos(angle_i), sin(angle_i));
        }
        lim++;
    }
}

void fft(vector<base> &a, int n = -1) {
    if(n == -1) n = a.size();
    assert((n & (n - 1)) == 0);
    int zeros = __builtin_ctz(n);
    ensure_base(zeros);
    int shift = lim - zeros;
    for(int i = 0; i < n; i++) if(i < (rev[i] >> shift)) swap(a[i], a[rev[i] >>
shift]);
    for(int k = 1; k < n; k <= 1) {
        for(int i = 0; i < n; i += 2 * k) {
            for(int j = 0; j < k; j++) {
                base z = a[i + j + k] * roots[j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] = a[i + j] + z;
            }
        }
    }
}

//eq = 0: 4 FFTs in total
//eq = 1: 3 FFTs in total
vector<int> multiply(vector<int> &a, vector<int> &b, int eq = 0) {
    int need = a.size() + b.size() - 1;
    int p = 0;
    while((1 << p) < need) p++;
    ensure_base(p);
    int sz = 1 << p;
    vector<base> A, B;
    if(sz > (int)A.size()) A.resize(sz);
    for(int i = 0; i < (int)a.size(); i++) {
        int x = (a[i] % mod + mod) % mod;
        A[i] = base(x & ((1 << 15) - 1), x >> 15);
    }
}

```

```

}
fill(A.begin() + a.size(), A.begin() + sz, base{0, 0});
fft(A, sz);
if(sz > (int)B.size()) B.resize(sz);
if(eq) copy(A.begin(), A.begin() + sz, B.begin());
else {
    for(int i = 0; i < (int)b.size(); i++) {
        int x = (b[i] % mod + mod) % mod;
        B[i] = base(x & ((1 << 15) - 1), x >> 15);
    }
    fill(B.begin() + b.size(), B.begin() + sz, base{0, 0});
    fft(B, sz);
}
double ratio = 0.25 / sz;
base r2(0, -1), r3(ratio, 0), r4(0, -ratio), r5(0, 1);
for(int i = 0; i <= (sz >> 1); i++) {
    int j = (sz - i) & (sz - 1);
    base a1 = (A[i] + conj(A[j])), a2 = (A[i] - conj(A[j])) * r2;
    base b1 = (B[i] + conj(B[j])) * r3, b2 = (B[i] - conj(B[j])) * r4;
    if(i != j) {
        base c1 = (A[j] + conj(A[i])), c2 = (A[j] - conj(A[i])) * r2;
        base d1 = (B[j] + conj(B[i])) * r3, d2 = (B[j] - conj(B[i])) * r4;
        A[i] = c1 * d1 + c2 * d2 * r5;
        B[i] = c1 * d2 + c2 * d1;
    }
    A[j] = a1 * b1 + a2 * b2 * r5;
    B[j] = a1 * b2 + a2 * b1;
}
fft(A, sz); fft(B, sz);
vector<int> res(need);
for(int i = 0; i < need; i++) {
    long long aa = A[i].x + 0.5;
    long long bb = B[i].x + 0.5;
    long long cc = A[i].y + 0.5;
    res[i] = (aa + ((bb % mod) << 15) + ((cc % mod) << 30))%mod;
}
return res;
}

vector<int> pow(vector<int>& a, int p) {
    vector<int> res;
    res.emplace_back(1);
    while(p) {
        if(p & 1) res = multiply(res, a);
        a = multiply(a, a, 1);
        p >>= 1;
    }
}

```

```

    }
    return res;
}
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(10, 0);
    while(k--) {
        int m; cin >> m;
        a[m] = 1;
    }
    vector<int> ans = pow(a, n / 2);
    int res = 0;
    for(auto x: ans) res = (res + 1LL * x * x % mod) % mod;
    cout << res << '\n';
    return 0;
}
//https://codeforces.com/contest/1096/problem/G

```

POW

```

vector<int> poly_pow(vector<int> poly, int p, int limit = 1e9) {
    vector<int> ans{1};
    while (p) {
        if(p&1) ans = conv(ans, poly);
        poly = conv(poly, poly);
        ans.resize(limit + 1);
        poly.resize(limit + 1);
        p >>= 1;
    }
    return ans;
}

```